

Principios básicos de filtrado y Transformada Z

Elaborado por Santiago Tabares satabaresgi@unal.edu.co, Andres Alvarez amalvarezme@unal.edu.co

Transformada de Laplace

La transformada de Laplace se puede entender como la transformada de Fourier de una función multiplicada por una exponencial decreciente, buscando cumplir las condición débil de Dirichlet $\int_T |x(t)| dt < \infty$.

Transformada de Fourier de la señal $x(t)$:

$$X(\omega) = \int_{-\infty}^{+\infty} x(t) e^{-j\omega t} dt$$

Si se multiplica la señal $x(t)$ por la exponencial $e^{-\sigma t}$ y se le halla su transformada de Fourier, se obtiene la transformada de Laplace:

$$\begin{aligned} \hat{x}(t, \sigma) &= x(t) e^{-\sigma t} \\ \hat{X}(\sigma, \omega) &= \int_{-\infty}^{+\infty} \hat{x}(t, \sigma) e^{-j\omega t} dt = \int_{-\infty}^{+\infty} x(t) e^{-\sigma t} e^{-j\omega t} dt \end{aligned}$$

Lo común es escribir ambas variables en una sola, haciendo $\sigma + j\omega = s$:

$$\begin{aligned} \hat{X}(\omega, \sigma) &= X(s) \\ X(s) &= \int_{-\infty}^{+\infty} x(t) e^{-st} dt \\ \mathcal{L}\{x(t)\} &= \mathcal{F}\{x(t) e^{-\sigma t}\} \end{aligned}$$

Transformada Z

Para extender el concepto de transformada de Laplace a señales discretas, se modela una señal $x_m(t)$ como una señal continua $x(t)$ muestreada por un *ideal sampler* (tren de impulsos de periodo T):

$$x_m(t) = \sum_{n=-\infty}^{+\infty} x(nT) \delta(t - nT)$$

Obteniendo la transformada de Laplace de esta señal:

$$\begin{aligned} X_m(s) &= \int_{-\infty}^{+\infty} \sum_{n=-\infty}^{+\infty} x(nT) \delta(t - nT) e^{-st} dt \\ X_m(s) &= \sum_{n=-\infty}^{+\infty} x(nT) \int_{-\infty}^{+\infty} \delta(t - nT) e^{-st} dt = \sum_{n=-\infty}^{+\infty} x(nT) e^{-snT} \end{aligned}$$

Notando que $x(nT)$ es $x[n]$ y haciendo $z = e^{sT}$ se obtiene la transformada Z:

$$\begin{aligned} X_m(s) &= X(z) \\ X(z) &= \sum_{n=-\infty}^{+\infty} x[n] z^{-n} \end{aligned}$$

✓ Relación entre el plano s y el plano z

La relación entre el plano s y el plano z puede obtenerse expresando s como número complejo:

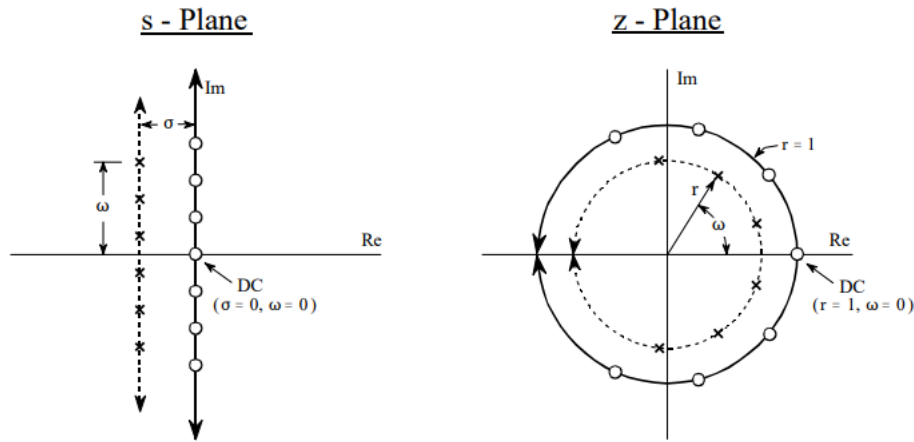
$$z = e^{sT} = e^{(\sigma + j\omega)T}$$

Al expresar $e^{\sigma T}$ como r y haciendo $\omega T = \Omega$ se obtiene la representación de z en forma polar:

$$z = r e^{j\Omega}$$

Es fundamental notar que los valores r y Ω dependen del periodo de muestreo T .

Como r depende de σ , líneas verticales en s mapean circunferencias en z , mientras que líneas horizontales en s (ω constante) mapean líneas rectas inclinadas en z .



Relationship between the s-plane and the z-plane. The s-plane is a rectangular coordinate system with σ expressing the distance along the real (horizontal) axis, and ω the distance along the imaginary (vertical) axis. In comparison, the z-plane is in polar form, with r being the distance to the origin, and ω the angle measured to the positive horizontal axis. Vertical lines in the s-plane, such as illustrated by the example poles and zeros in this figure, correspond to circles in the z-plane.

- Recordar que la transformada de Fourier se obtiene cuando $\sigma = 0$, es decir, evaluando s sobre el eje vertical, quedando $s = j\omega$.
- De manera similar, debido a la dependencia de r con respecto a σ , la transformada en tiempo discreto de Fourier (DTFT) se obtiene cuando $r = 1$, es decir, evaluando z sobre la circunferencia unitaria, quedando $z = e^{j\Omega}$.

Por este motivo, es que el espectro de una señal discreta $x[n]$ se suele denotar como $X(e^{j\Omega})$.

Modelado con la transformada Z

De manera similar, al modelado mediante ecuaciones diferenciales de los sistemas continuos, un SLIT discreto puede modelarse mediante ecuaciones de diferencias:

$$a_0 y[n] + a_1 y[n-1] + a_2 y[n-2] + \cdots + a_N y[n-N] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2] + \cdots + b_M x[n-M]$$

$$\sum_{k=0}^N a_k y[n-k] = \sum_{k=0}^M b_k x[n-k]$$

Aplicando la transformada Z a esta expresión:

$$a_0 Y(z) + a_1 z^{-1} Y(z) + a_2 z^{-2} Y(z) + \cdots + a_N z^{-N} Y(z) = b_0 X(z) + b_1 z^{-1} X(z) + b_2 z^{-2} X(z) + \cdots + b_M z^{-M} X(z)$$

Nota: Se sabe que $\mathcal{Z}\{x[n \pm M]\} = z^{\pm M} X(z)$.

La transformada Z plantea que:

$$X(z) = \sum_{n=-\infty}^{+\infty} x[n] z^{-n}$$

Así:

$$\mathcal{Z}\{x[n \pm M]\} = \sum_{n=-\infty}^{+\infty} x[n \pm M] z^{-n}$$

Para $n' = n \pm M$; $n = n' \mp M$:

$$\sum_{n'=-\infty}^{+\infty} x[n'] z^{-(n' \mp M)} = \sum_{n'=-\infty}^{+\infty} x[n'] z^{-n'} z^{\pm M}$$

$$z^{\pm M} \sum_{n'=-\infty}^{+\infty} x[n'] z^{-n'} = z^{\pm M} X(z)$$

Por ende,

$$\left(\sum_{k=0}^N a_k z^{-k} \right) Y(z) = \left(\sum_{k=0}^M b_k z^{-k} \right) X(z)$$

Al hallar la función de transferencia se observa que un SLIT discreto puede modelarse como un cociente de polinomios en z :

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^M b_k z^{-k}}{\sum_{k=0}^N a_k z^{-k}}$$

Sistemas no recursivos (FIR)

Un sistema que carece de realimentación se denomina sistema de *respuesta finita al impulso* (FIR). Al carecer de realimentación, se tiene que $a_k = 0, \forall k = \{1, 2, 3, \dots, N\}$, y $a_0 = 1$. De modo que su función de transferencia es (solo ceros):

$$H(z) = \frac{Y(z)}{X(z)} = \sum_{k=0}^M b_k z^{-k}$$

Sistemas recursivos (IIR)

La forma canónica directa de los sistemas de *respuesta infinita al impulso* (IIR) se obtiene haciendo $a_0 = 1$, con lo que su función de transferencia se define como:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{k=0}^M b_k z^{-k}}{1 - \sum_{k=1}^N a_k z^{-k}}$$

✓ Diseño de Filtros IIR

Invarianza del impulso

Este método de diseño se basa en el muestreo de la respuesta impulso de un filtro en tiempo continuo $h_c(t)$, multiplicada por el período de muestreo T_s :

$$h[n] = T_s h_c(nT_s).$$

Un sistema continuo se puede representar en s como una suma de fracciones parciales:

$$H_c(s) = \sum_{k=1}^N \frac{A_k}{s - s_k}.$$

$$\frac{Y(s)}{X(s)} = H(s),$$

$$x(t) = \delta(t)$$

$$X(s) = 1$$

La correspondiente función en tiempo es entonces (Ver [tablas transformada de Laplace](#)):

$$h_c(t) = \begin{cases} \sum_{k=1}^N A_k e^{s_k t} & t \geq 0 \\ 0 & t < 0 \end{cases}$$

Realizando el muestreo:

$$h[n] = T_s \sum_{k=1}^N A_k e^{s_k n T_s} u[n].$$

Al obtener la transformada Z (Ver [tablas transformada Z](#)), se obtiene:

$$H(z) = \sum_{k=1}^N \frac{T_s A_k}{1 - e^{s_k T_s} z^{-1}}$$

De manera que conociendo los polos $s = s_k$ se puede obtener los polos $z = e^{s_k T_s}$.

Ejemplo de diseño de un filtro en tiempo discreto a partir de su contraparte en tiempo continuo

Supóngase que se tiene un filtro con los siguientes polos, queriendo realizar un filtrado con $T_s = 1$:

$$s_{1,2} = -\frac{1}{2} \pm j\frac{1}{2}$$

La correspondiente función de transferencia en s se encuentra según sus polos como:

$$H_c(s) = \frac{j}{s + \frac{1+j}{2}} - \frac{j}{s + \frac{1-j}{2}}$$

Los polos en z se calculan de la manera indicada anteriormente

$$z_1 = e^{-\frac{1+j}{2}}$$

$$z_2 = e^{-\frac{1-j}{2}}$$

Finalmente, obteniendo la función de transferencia en z

$$H(z) = \frac{j}{1 - e^{-\frac{1+j}{2}} z^{-1}} - \frac{j}{1 - e^{-\frac{1-j}{2}} z^{-1}}$$

$$H(z) = \frac{jz}{z - e^{-\frac{1+j}{2}}} - \frac{jz}{z - e^{-\frac{1-j}{2}}}$$

Transformación bilineal

Este método consiste en realizar el siguiente mapeo (Ver aproximación función logartimo en [transformación bilineal](#))

$$s = \frac{2}{T_s} \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right)$$

Al evaluar $H_c(s)$ en tal expresión, se obtiene la función de transferencia en z

$$H(z) = H_c \left(\frac{2}{T_s} \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right) \right)$$

El mapeo correspondiente de s a z se obtiene despejando z de la primera expresión

$$z = \frac{1 + (T_s/2)s}{1 - (T_s/2)s}$$

Ejemplo

Supongase que se tiene el mismo filtro anterior, pero esta vez se desea obtener la expresión en z mediante transformación bilinear.

Realizando la evaluación de s como se expuso

$$H_c \left(2 \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right) \right) = \frac{j}{2 \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right) + \frac{1+j}{2}} - \frac{j}{2 \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right) + \frac{1-j}{2}}$$

Desarrollando algebraicamente se llega finalmente a la expresión

$$H(z) = \frac{1 + 5j}{13} \cdot \frac{z + 1}{z + \frac{-7+4j}{13}} + \frac{1 - 5j}{13} \cdot \frac{z + 1}{z - \frac{7+4j}{13}}$$

Que es precisamente la función de transferencia que se quería encontrar.

Este último es el método que se suele usar para diseñar aproximaciones de los filtros Butterworth, Chebyshev y elípticos en tiempo discreto.

✓ Señal de audio

Audio a procesar. Se puede realizar con cualquier vídeo de youtube usando youtube-dl, únicamente se necesita el enlace al vídeo (pueden introducir el que ustedes quieran). Debería quedar guardado en el folder como audio.mp3

```
1 #! pip install youtube-dl
2 !python3 -m pip install --force-reinstall https://github.com/yt-dlp/yt-dlp/archive/master.tar.gz
```

```
Collecting https://github.com/yt-dlp/yt-dlp/archive/master.tar.gz
  Downloading https://github.com/yt-dlp/yt-dlp/archive/master.tar.gz (2.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.8/2.8 MB 11.9 MB/s eta 0:00:00
Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: yt-dlp
  Building wheel for yt-dlp (pyproject.toml) ... done
```

```
Created wheel for yt-dlp: filename=yt_dlp-2025.6.30-py3-none-any.whl size=3017662 sha256=111fbd155e3fea8f05c9d8ebf7d70210c4060058f9fef
Stored in directory: /tmp/pip-ephem-wheel-cache-pdaz1f73/wheels/2d/79/97/7209650ef73114e0fe0603480da012ad3afac9cae6b8acd9a
Successfully built yt-dlp
Installing collected packages: yt-dlp
Successfully installed yt-dlp-2025.6.30
```

```
1 link="https://www.youtube.com/watch?v=M-43pOqheMY&ab_channel=MetalBladeRecords"
2 !yt-dlp --extract-audio -o "audio" --audio-format mp3 {link}
```

```
[youtube] Extracting URL: https://www.youtube.com/watch?v=M-43pOqheMY
[youtube] M-43pOqheMY: Downloading webpage
[youtube] M-43pOqheMY: Downloading tv client config
[youtube] M-43pOqheMY: Downloading player e12fba4-main
[youtube] M-43pOqheMY: Downloading tv player API JSON
[youtube] M-43pOqheMY: Downloading ios player API JSON
[youtube] M-43pOqheMY: Downloading m3u8 information
[info] M-43pOqheMY: Downloading 1 format(s): 251
[download] Destination: audio
[download] 100% of 4.03MiB in 00:00:00 at 28.63MiB/s
[ExtractAudio] Destination: audio.mp3
Deleting original file audio (pass -k to keep)
```

```
1 !ffmpeg -i audio.mp3 output.wav
```

```
ffmpeg version 4.4.2-0ubuntu0.22.04.1 Copyright (c) 2000-2021 the FFmpeg developers
built with gcc 11 (Ubuntu 11.2.0-19ubuntu1)
configuration: --prefix=/usr --extra-version=0ubuntu0.22.04.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/in
libavutil 56. 70.100 / 56. 70.100
libavcodec 58.134.100 / 58.134.100
libavformat 58. 76.100 / 58. 76.100
libavdevice 58. 13.100 / 58. 13.100
libavfilter 7.110.100 / 7.110.100
libswscale 5. 9.100 / 5. 9.100
libswresample 3. 9.100 / 3. 9.100
libpostproc 55. 9.100 / 55. 9.100
Input #0, mp3, from 'audio.mp3':
Metadata:
  encoder      : Lavf58.76.100
Duration: 00:04:32.83, start: 0.023021, bitrate: 135 kb/s
Stream #0:0: Audio: mp3, 48000 Hz, stereo, fltp, 135 kb/s
Metadata:
  encoder      : Lavc58.13
Stream mapping:
  Stream #0:0 -> #0:0 (mp3 (mp3float) -> pcm_s16le (native))
Press [q] to stop, [?] for help
Output #0, wav, to 'output.wav':
Metadata:
  ISFT        : Lavf58.76.100
Stream #0:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 48000 Hz, stereo, s16, 1536 kb/s
Metadata:
  encoder      : Lavc58.134.100 pcm_s16le
size= 51150kB time=00:04:32.78 bitrate=1536.1kbits/s speed= 306x
video:0kB audio:51150kB subtitle:0kB other streams:0kB global headers:0kB muxing overhead: 0.000149%
```

```
1 !pip install soundfile
```

```
Requirement already satisfied: soundfile in /usr/local/lib/python3.11/dist-packages (0.13.1)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.11/dist-packages (from soundfile) (1.17.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from soundfile) (2.0.2)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/dist-packages (from cffi>=1.0->soundfile) (2.22)
```

```
1 import soundfile as sf # para instalar pip install soundfile
2 #lee archivos wav
3 nombre_out = "output.wav"
4 x, fs = sf.read(nombre_out)
5
6 # read speech signal from file
7 print('Frecuencia de muestreo %.2f[Hz]\naudio %s' % (fs,nombre_out))
```

```
Frecuencia de muestreo 48000.00[Hz]
audio output.wav
```

```
1 x.shape[0]/fs #segundos de la canción
```

```
272.79964583333333
```

```

1 xpro = x.copy() #copiar archivos para procesar
2 ti = 30
3 tf = 50
4 xs = xpro[int(ti*fs):int((tf*fs)),:]

1 #Para escuchar el trozo de audio con el que se va a trabajar:
2 #No usar un trozo muy grande o el entorno se puede desconectar
3 from IPython.display import Audio
4 Audio([xs[:,1], xs[:,0]],rate=fs)

```



0:00 / 0:20



✓ Filtrado y el retardo de fase

- Dado que el filtro digital se puede entender como un sistema SLIT, su respuesta al impulso genera un desfase en la frecuencia que se evidencia como un retardo en el tiempo. El objetivo es mantener dicho retardo estable y conocido para compensarlo.

$$\mathfrak{F}\{x(t \pm t_0)\} = X(w)e^{\mp j\omega t_0}$$

$$X(w) = |X(w)|e^{j\theta}$$

$$y(t) = \mathfrak{H}\{x(t)\} = h(t) * x(t)$$

$$Y(w) = X(w)H(w) = |X(w)||H(w)|e^{j(\theta_x + \theta_h)}$$

- Algunas alternativas para compensar el retarda en fase incluyen la estimación del mismo desde los puntos en el tiempo (ver [lfitler_zi](#)) o la evaluación del filtro hacia adelante y hacia atrás (ver [filtfilt](#)).

✓ Filtro Butterworth

El [filtro Butterworth](#) posee la ventaja de contar con la respuesta más plana posible para la banda pasante, sin embargo la banda de transición es muy amplia.

```

1 from scipy.signal import butter as bw
2 from scipy.signal import freqz_zpk

1 N = 1 #orden el filtro
2 Wn = [2000, 5000] #banda pasante
3 filt = 'bandpass' #tipo del filtro (pasa-baja, pasa-alta, pasabanda, rechazabanda)
4 out = 'zpk' #la funcion retorna los polos y ceros junto con la ganancia. Para usar numerador y denominador usar 'ba' junto con freqz
5 zeros, poles, gain = bw(N, Wn, btype=filt, output=out, fs=fs)
6 #K = len(Xw[:,0]) #longitud del vector transformada
7 w, h = freqz_zpk(zeros, poles, gain, fs=fs) #Para indicar la longitud usar worN=K
8

1 import numpy as np
2 from matplotlib import pyplot as plt
3 from matplotlib.patches import Circle
4
5 def plot_freq_response(filter_name, w, h, N): #Mostrar la respuesta en frecuencia del filtro
6     fig = plt.figure()
7     ax1 = fig.add_subplot(1, 1, 1)
8     ax1.set_title(filter_name + ' Digital filter frequency response, order= ' + str(N))
9     ax1.plot(w, 20 * np.log10(abs(h)), 'b')
10    ax1.set_ylabel('Amplitude [dB]', color='b')
11    ax1.set_xlabel('Frequency [Hz]')
12    ax1.grid()
13    ax2 = ax1.twinx()
14    angles = np.unwrap(np.angle(h))
15    ax2.plot(w, angles, 'g')
16    ax2.set_ylabel('Angle [radians]', color='g')
17    plt.axis('tight')
18    plt.show()

1 def show_zp(z, p, title= 'Z-plane'): # Mostrar la ubicación de los polos y los zeros
2     ax = plt.gca()
3
4     ax.plot(np.real(z), np.imag(z), 'bo', fillstyle='none', ms = 10)

```

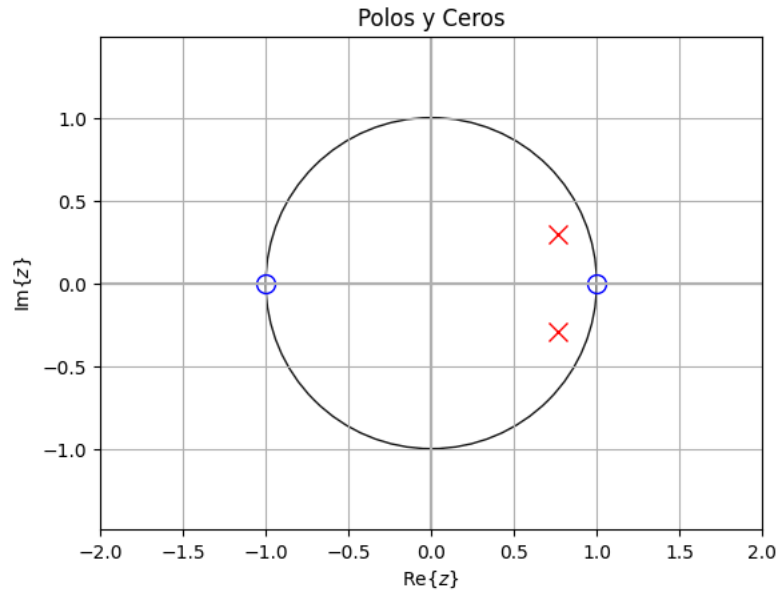
```

5 ax.plot(np.real(p), np.imag(p), 'rx', fillstyle='none', ms = 10)
6 unit_circle = Circle((0,0), radius=1, fill=False,
7                       color='black', ls='solid', alpha=0.9)
8 ax.add_patch(unit_circle)
9 ax.axvline(0, color='0.7')
10 ax.axhline(0, color='0.7')
11
12 plt.title(title)
13 plt.xlabel(r'Re{$z$}')
14 plt.ylabel(r'Im{$z$}')
15 plt.axis('equal')
16 plt.xlim((-2, 2))
17 plt.ylim((-2, 2))
18 plt.grid()

```

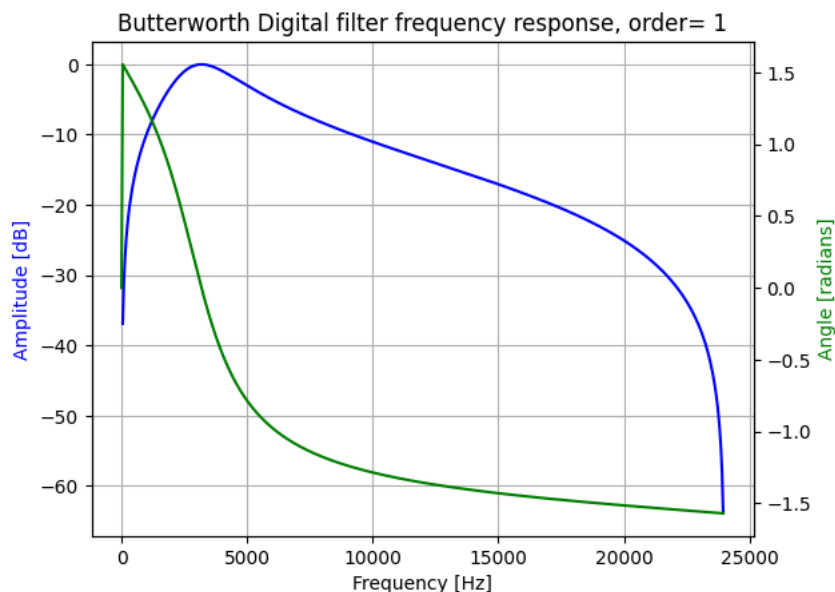
```
1 show_zp(zeros, poles, title='Polos y Ceros')
```

⚠ WARNING:matplotlib.axes._base:Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.



```
1 plot_freq_response('Butterworth', w, h, N)
```

⚠ /tmp/ipython-input-11-2558837945.py:9: RuntimeWarning: divide by zero encountered in log10
ax1.plot(w, 20 * np.log10(abs(h)), 'b')



```

1 #Puede obtenerse un mejor resultado aumentando el grado del filtro, a cambio de un mayor coste computacional y problemas en la fase
2 N = 4
3 zeros, poles, gain = bw(N, Wn, btype='lowpass', output='zpk', fs=fs)

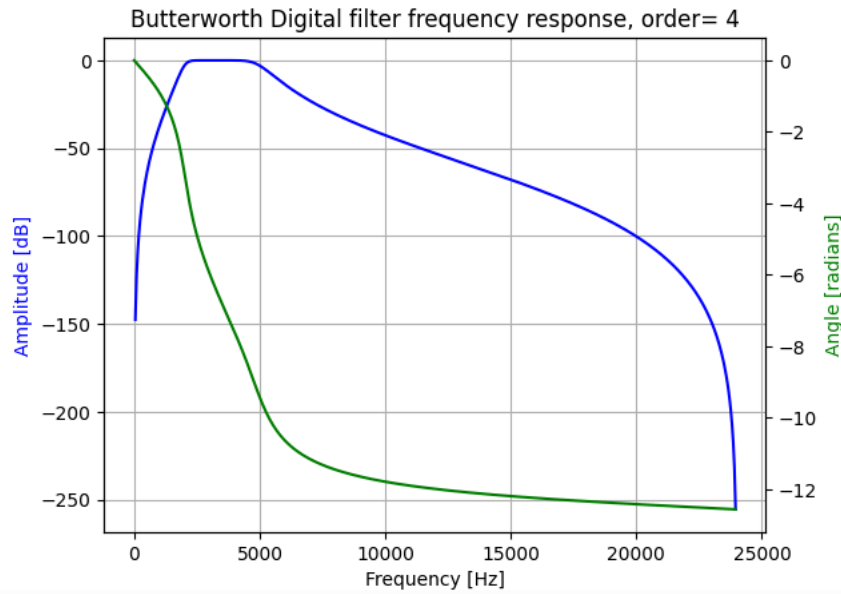
```

```

4 w, h = freqz_zpk(zeros, poles, gain, fs=fs)
5
6 plot_freq_response('Butterworth', w, h, N)

```

→ /tmp/ipython-input-11-2558837945.py:9: RuntimeWarning: divide by zero encountered in log10
 ax1.plot(w, 20 * np.log10(abs(h)), 'b')



✓ Filtro Chebyshev

Al contrario del filtro Butterworth, el filtro de [Chebyshev](#) tiene una pendiente en la caída muy pronunciada, por lo que la banda de transición resulta ser menor, a cambio de que existe un rizado en la banda pasante y/o rechazada. Dependiendo del lugar en el que se escoja el rizado, se tienen dos filtros diferentes de Chebyshev.

✓ Chebyshev 1

Para este filtro se busca un mayor rechazo en la banda rechazada a cambio de obtener un rizado en la banda pasante

```

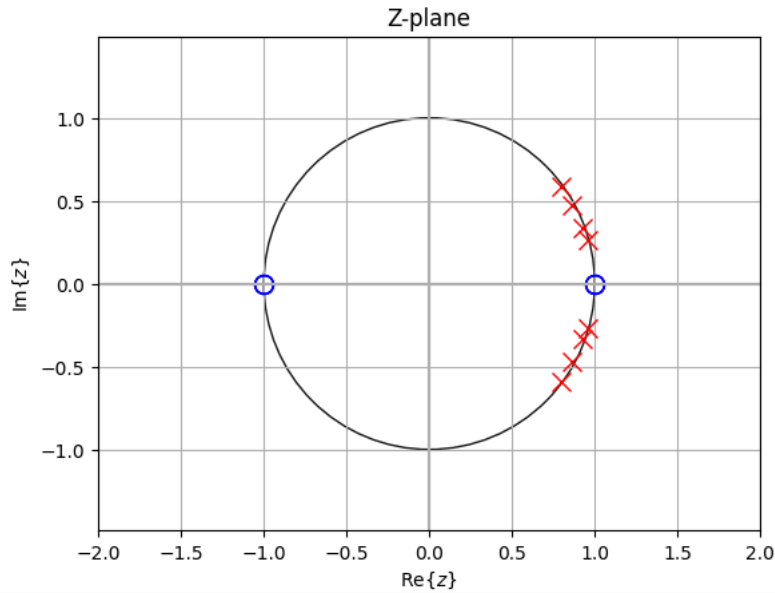
1 from scipy.signal import cheby1

1 N = 4
2 ripple = 10
3 zeros, poles, gain = cheby1(N, rp=ripple, Wn=Wn, btype=filt, output=out, fs=fs)
4 w, h = freqz_zpk(zeros, poles, gain, fs=fs)

1 show_zp(zeros, poles)

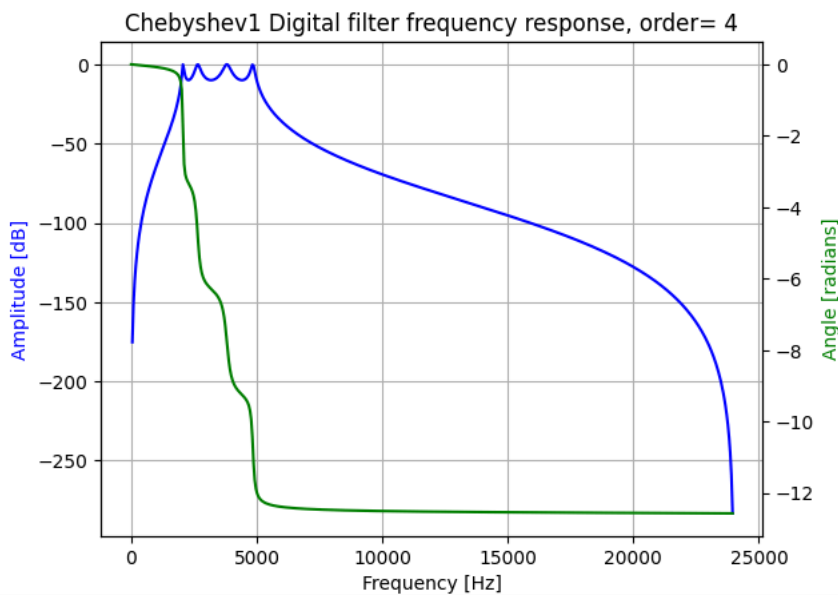
```

WARNING:matplotlib.axes._base:Ignoring fixed y limits to fulfill fixed data aspect with adjustable data limits.



```
1 plot_freq_response('Chebyshev1', w, h, N)
```

/tmp/ipython-input-11-2558837945.py:9: RuntimeWarning: divide by zero encountered in log10
ax1.plot(w, 20 * np.log10(abs(h)), 'b')



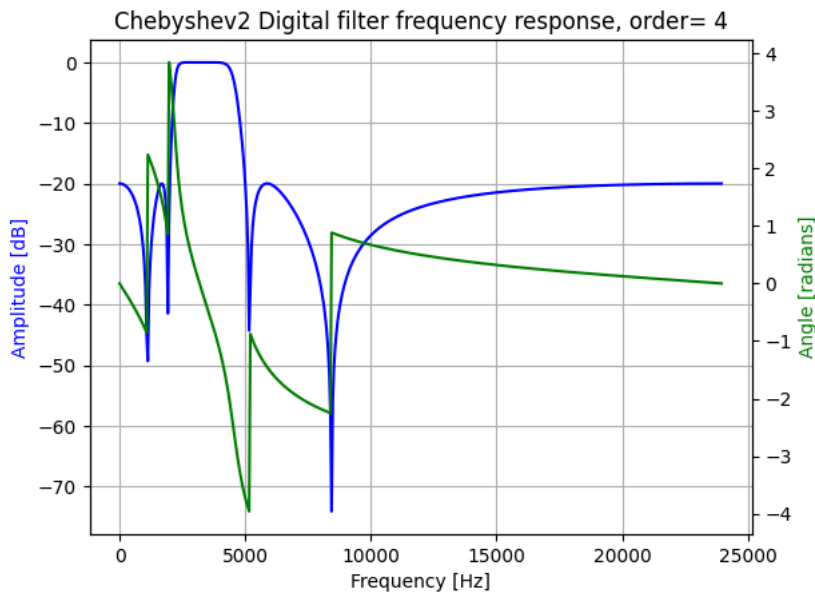
✓ Chebyshev 2

Para este caso del filtro Chebyshev, se prefiere que se tenga una respuesta plana en la banda pasante, obteniendo ahora el rizo en la banda rechazada

```
1 from scipy.signal import cheby2
2 from scipy.signal import freqz #para este caso no se van a mostrar los polos y ceros

1 N = 4
2 ripple = 20
3 out = 'ba' #no regresa los polos y ceros, sino la funcion de transferencia
4 num, den = cheby2(N, rs=ripple, Wn=Wn, btype='lowpass', output=out, fs=fs) #regresa numerador y denominador. Si se quiere obtener los polos y
5 w, h = freqz(num, den, fs=fs)
```

```
1 plot_freq_response('Chebyshev2', w, h, N)
```



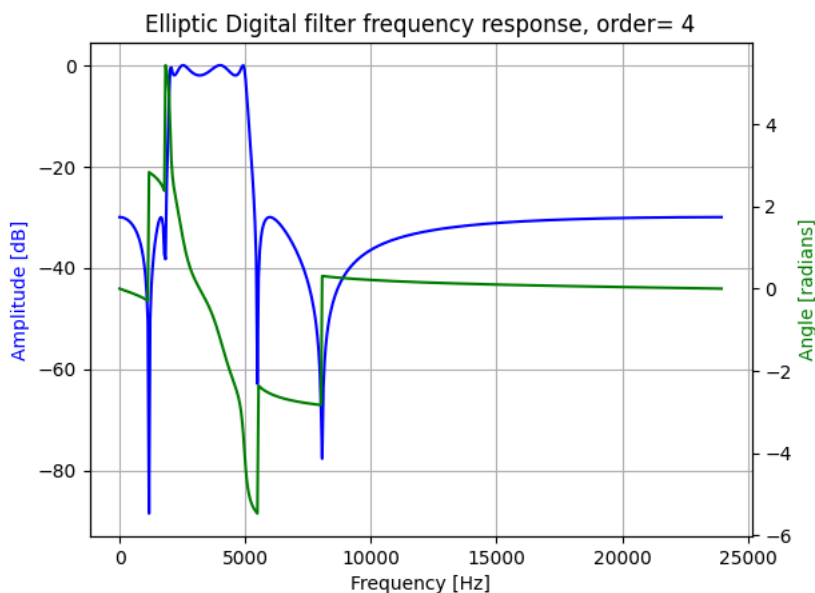
✓ Filtro Elíptico (Filtro de Cauér)

Este filtro [Elíptico](#) es el que contiene la banda de transición más delgada posible, incluyendo además rizado tanto en la banda pasante como en la rechazada (ambos rizados son independientes entre sí)

```
1 from scipy.signal import ellip

1 N = 4
2 ripple_pass = 2 #para la banda pasante
3 ripple_stop = 30 #para la banda rechazada
4 num, den = ellip(N, rp=ripple_pass, rs=ripple_stop, Wn=Wn, btype=filt, output=out, fs=fs) #regresa numerador y denominador. Si se quiere
5 w, h = freqz(num, den, fs=fs)

1 plot_freq_response('Elliptic', w, h, N)
```



✓ Filtro de Bessel

El filtro de [Bessel](#) posee una respuesta casi lineal en fase.

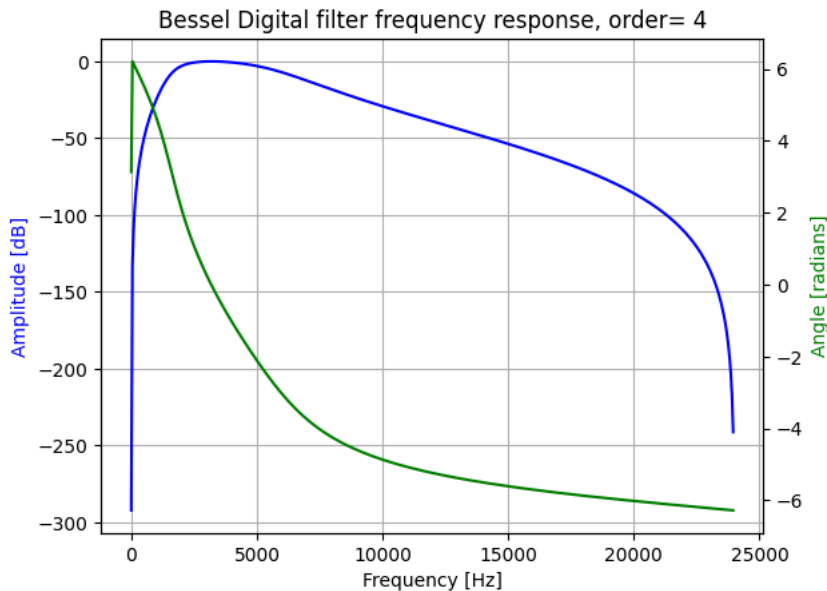
```
1 from scipy.signal import bessel
```

```

1 N = 4
2 normalization = 'mag' #Ajusta la frecuencia crítica según la respuesta que se desea
3 num, den = bessell(N, Wn=Wn, btype=filt, norm=normalization, output=out, fs=fs) #regresa numerador y denominador. Si se quiere obtener lc
4 w, h = freqz(num, den, fs=fs)

```

```
1 plot_freq_response('Bessel', w, h, N)
```



✓ Ecualización

Entre las diversas aplicaciones que tienen los filtros está el procesado de audio. Los dispositivos que ayudan a realizar estas tareas son los llamados ecualizadores, con los que se pueden realzar sonidos a unas determinadas frecuencias o suprimirlos si se desea.

```

1 #Audio sobre el que se va a trabajar para este ejemplo.
2 link="https://www.youtube.com/watch?v=_ghCnV_z7Vc"
3 !yt-dlp --extract-audio -o "audio2" --audio-format mp3 {link}
4
5 !ffmpeg -i audio2.mp3 output2.wav
6
7 nombre_out = "output2.wav"
8 x, fs = sf.read(nombre_out)

```



```

[youtube] Extracting URL: https://www.youtube.com/watch?v=_ghCnV_z7Vc
[youtube] _ghCnV_z7Vc: Downloading webpage
[youtube] _ghCnV_z7Vc: Downloading tv client config
[youtube] _ghCnV_z7Vc: Downloading tv player API JSON
[youtube] _ghCnV_z7Vc: Downloading ios player API JSON
[youtube] _ghCnV_z7Vc: Downloading m3u8 information
[info] _ghCnV_z7Vc: Downloading 1 format(s): 251
[download] Destination: audio2
[download] 100% of 3.18MiB in 00:00:00 at 13.00MiB/s
[ExtractAudio] Destination: audio2.mp3
Deleting original file audio2 (pass -k to keep)
ffmpeg version 4.4.2-0ubuntu0.22.04.1 Copyright (c) 2000-2021 the FFmpeg developers
built with gcc 11 (Ubuntu 11.2.0-19ubuntu1)
configuration: --prefix=/usr --extra-version=0ubuntu0.22.04.1 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/in
libavutil 56. 70.100 / 56. 70.100
libavcodec 58.134.100 / 58.134.100
libavformat 58. 76.100 / 58. 76.100
libavdevice 58. 13.100 / 58. 13.100
libavfilter 7.110.100 / 7.110.100
libswscale 5. 9.100 / 5. 9.100
libswresample 3. 9.100 / 3. 9.100
libpostproc 55. 9.100 / 55. 9.100
Input #0, mp3, from 'audio2.mp3':
Metadata:
encoder : Lavf58.76.100
Duration: 00:03:28.82, start: 0.023021, bitrate: 71 kb/s
Stream #0:0: Audio: mp3, 48000 Hz, stereo, fltp, 71 kb/s
Metadata:
encoder : Lavc58.13

```

```
Stream mapping:
  Stream #0:0 -> #0:0 (mp3 (mp3float) -> pcm_s16le (native))
Press [q] to stop, [?] for help
Output #0, wav, to 'output2.wav':
Metadata:
  ISFT              : Lavf58.76.100
  Stream #0:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 48000 Hz, stereo, s16, 1536 kb/s
Metadata:
  encoder           : Lavc58.134.100 pcm_s16le
size= 39147kB time=00:03:28.77 bitrate=1536.0kbits/s speed= 488x
video:0kB audio:39147kB subtitle:0kB other streams:0kB global headers:0kB muxing overhead: 0.000195%
```

```
1 x2 = x.copy()
2 ti = 63
3 tf = ti + 6
4 xs = x2[int(ti*fs):int(tf*fs)],:]
5 K= int(len(xs)/2) + 1
6
7 Audio([xs[:,1], xs[:,0]],rate=fs) #segmento de audio
```



✓ Filtro paso alto

En los instrumentos musicales existe una banda de frecuencias en los que se pueden tocar, mientras que en otros les es imposible. Por este motivo es que es posible suprimir algunos instrumentos de una pista mediante el uso de filtros.

Como ejemplo, supongase que en el audio anterior se quiere suprimir el sonido del bajo eléctrico. Como su nombre lo indica, este es un instrumento que se caracteriza por manejar una banda de frecuencias bajas (sonidos graves), por lo que aplicando un filtro de paso alto se podría realizar esta tarea.

```
1 filtro = "highpass"
2 N = 6
3 freq = 1000
4
5 ripple_pass = 2
6 ripple_stop = 101
7 num, den = ellip(N, rp=ripple_pass, rs=ripple_stop, Wn=freq, btype=filtro, fs=fs)

1 from scipy.signal import lfilter, filtfilt # filtfilt compensa el retardo en tiempo debido a la respuesta en fase del filtro

1 xf = filtfilt(num, den, xs, axis=0)

1 Audio([xf[:,1], xf[:,0]],rate=fs) #Resultado
```



✓ Filtro paso bajo

Un problema que le ocurre continuamente a las señales es la presencia de ruido. A continuación se muestra el mismo audio contaminado con ruido blanco

```
1 x_n = xs + np.random.normal(loc=0, scale=0.05, size=xs.shape)

1 Audio([x_n[:,1], x_n[:,0]],rate=fs) #Resultado
```



Por lo general el ruido se caracteriza por manejar una banda de frecuencias altas. Teniendo esto en cuenta, puede deshacerse de dicho ruido aplicando un filtro que suprima las frecuencias altas, es decir, un filtro pasabajas.

```

1 filtro = "lowpass"
2 N = 10
3 freq = 1600
4
5 ripple_pass = 1
6 ripple_stop = 100
7 num, den = ellip(N, rp=ripple_pass, rs=ripple_stop, Wn=freq, btype=filtro, fs=fs)
8
9 xf = lfilter(num, den, x_n, axis=0)

```

En el audio filtrado se percibe menos ruido, pero las frecuencias altas del audio original también se ven afectadas.

```
1 Audio([xf[:,1], xf[:,0]],rate=fs) #Audio con ruido filtrado
```



Según la aplicación de interés se pueden trabajar filtros pasabanda y rechazanda.

El pasabanda permite atenuar frecuencias por fuera de un rango determinado, mientras que el rechazabanda atenua un rango de frecuencias determinado.

✓ Ejercicios

1. Descargue 10 segundos de la canción de su preferencia desde youtube, y genere un filtro pasabanda (el usuario debe poder definir las frecuencias de corte) para cada uno de los filtros descritos (el usuario también debe poder fijar los parámetros de diseño de cada filtro). Compare los resultados de los filtros estudiados en este cuaderno tipo IIR para diseño Butterworth, Chebyshev 1, Chebyshev 2, Bessel y Elíptico.
2. Consulte en qué consiste el método de diseño de filtros FIR por ventaneo (Ver función [firwin](#) y [material de apoyo](#)). Realice un cuadro comparativo de las ventajas y desventajas de los filtros IIR y los FIR. Nota: Recuerde que un filtro FIR utiliza solamente raíces tipo ceros, es decir que $a_0 = 1$, y $a_k = 0 \forall k \in \{1, 2, \dots\}$.
3. Incluya la implementación del filtro `firwin` en el punto 1.

```

1 # 📦 Librerías necesarias
2 !pip install soundfile
3
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from scipy.signal import butter, cheby1, cheby2, ellip, bessel, filtfilt, freqz, freqz_zpk
7 import soundfile as sf
8 from IPython.display import Audio
9
10 # 🎧 1. Cargar audio (debes subir "audio.wav")
11 nombre_archivo = "output.wav"
12 x, fs = sf.read(nombre_archivo)
13 print(f"Frecuencia de muestreo: {fs} Hz, duración: {x.shape[0]/fs:.2f} s")
14
15 # ✂️ 2. Seleccionar 10 segundos desde el segundo 30
16 ti = 30
17 tf = ti + 10
18 xs = x[int(ti * fs):int(tf * fs), :]
19
20 # ⚙️ 3. Configuración del filtro
21 N = 4
22 Wn = [1000, 4000] # Banda pasante
23 filtro = 'bandpass'
24
25 # 🛠️ 4. Función para aplicar y mostrar el filtro
26 def aplicar_y_graficar(nombre, funcion, **kwargs):
27     print(f"\n 🎧 Filtro: {nombre}")
28     # Get 'ba' output for filtfilt
29     b, a = funcion(N, Wn=Wn, btype=filtro, fs=fs, **{k: v for k, v in kwargs.items() if k != 'output'})
30
31     # Get 'zpk' or 'ba' output for freq response plot based on original kwargs
32     if 'output' in kwargs and kwargs['output'] == 'zpk':
33         z, p, k = funcion(N, Wn=Wn, btype=filtro, fs=fs, **kwargs)

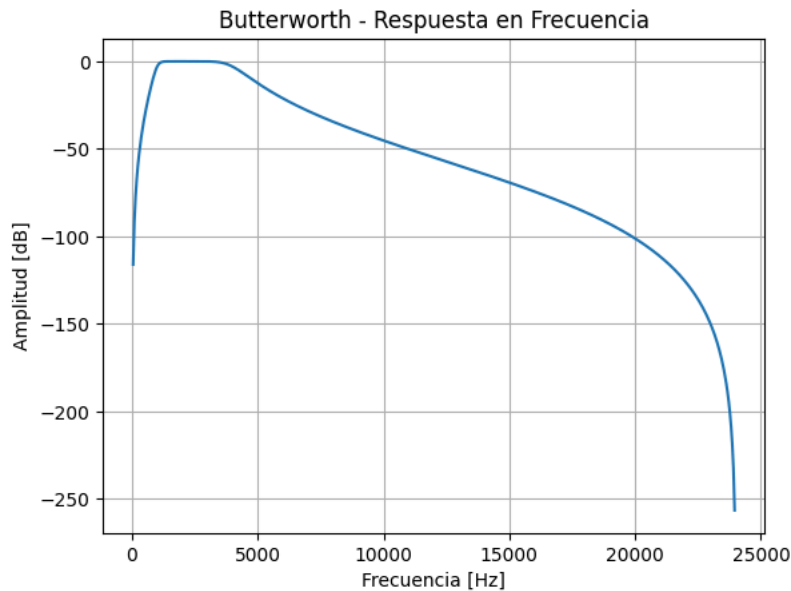
```

```
34     w, h = freqz_zpk(z, p, k, fs=fs)
35 else:
36     w, h = freqz(b, a, fs=fs)
37
38
39 # 📊 Gráfica de frecuencia
40 plt.figure()
41 plt.title(f"{nombre} - Respuesta en Frecuencia")
42 plt.plot(w, 20 * np.log10(abs(h)))
43 plt.ylabel("Amplitud [dB]")
44 plt.xlabel("Frecuencia [Hz]")
45 plt.grid()
46 plt.show()
47
48 # 🔊 Aplicar filtro
49 xfiltrado = filtfilt(b, a, xs, axis=0)
50 display(Audio([xfiltrado[:, 1], xfiltrado[:, 0]], rate=fs))
51
52 # 🚀 5. Aplicar todos los filtros
53 aplicar_y_graficar("Butterworth", butter, output='zpk')
54 aplicar_y_graficar("Chebyshev I", cheby1, rp=1, output='zpk')
55 aplicar_y_graficar("Chebyshev II", cheby2, rs=40)
56 aplicar_y_graficar("Elíptico", ellip, rp=1, rs=40)
57 aplicar_y_graficar("Bessel", bessel, norm='mag')
```

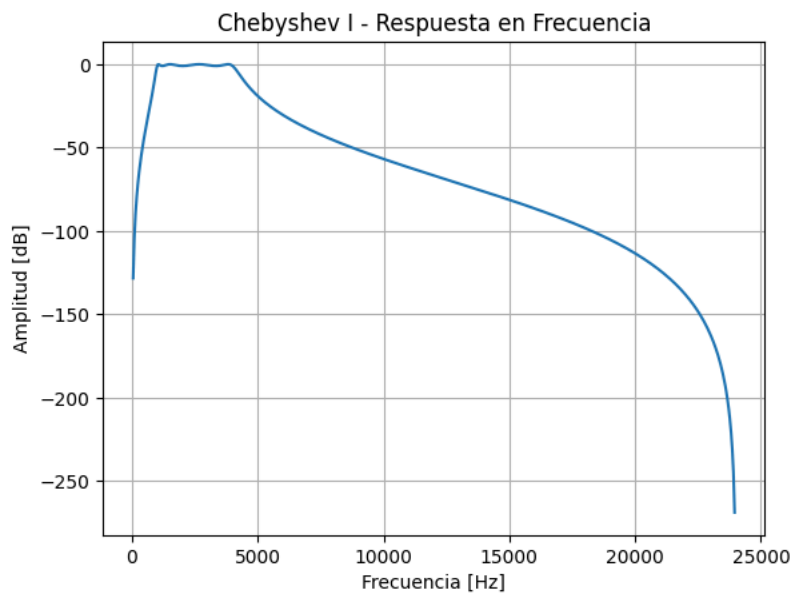
Requirement already satisfied: soundfile in /usr/local/lib/python3.11/dist-packages (0.13.1)
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.11/dist-packages (from soundfile) (1.17.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from soundfile) (2.0.2)
Requirement already satisfied: pycparser in /usr/local/lib/python3.11/dist-packages (from cffi>=1.0->soundfile) (2.22)
Frecuencia de muestreo: 48000 Hz, duración: 272.80 s

♦ Filtro: Butterworth

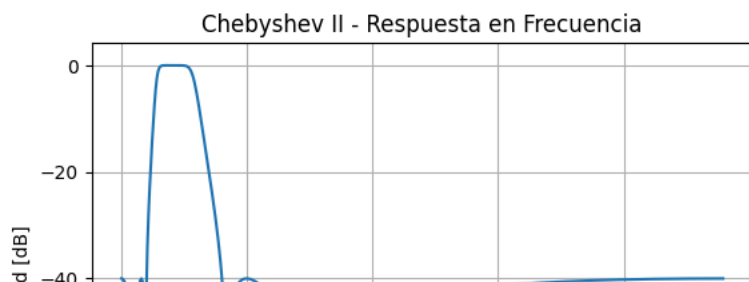
/tmp/ipython-input-42-3339804088.py:40: RuntimeWarning: divide by zero encountered in log10
plt.plot(w, 20 * np.log10(abs(h)))

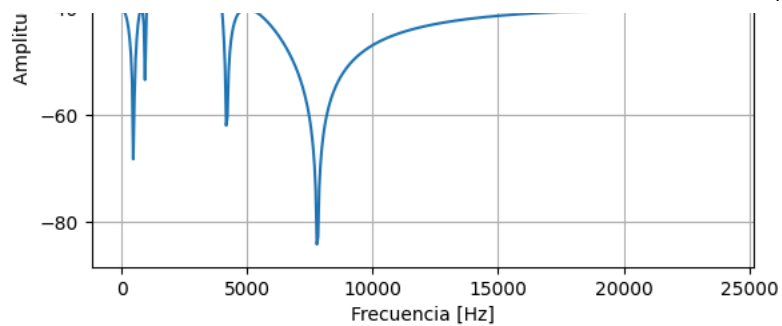


♦ Filtro: Chebyshev I



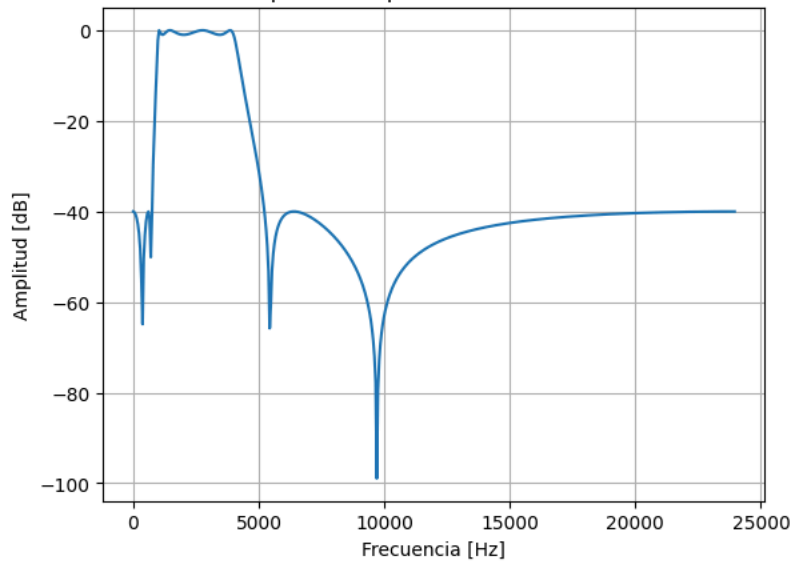
♦ Filtro: Chebyshev II





◆ Filtro: Elíptico

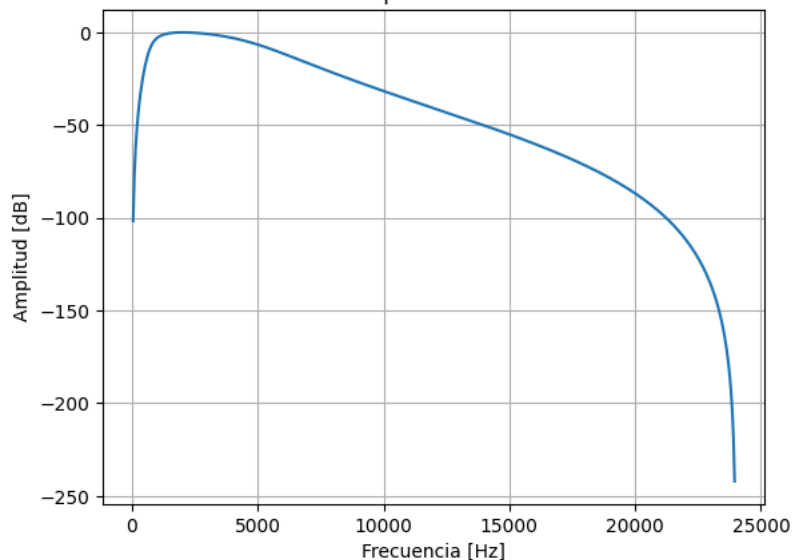
Elíptico - Respuesta en Frecuencia



◆ Filtro: Bessel

/tmp/ipython-input-42-3339804088.py:40: RuntimeWarning: divide by zero encountered in log10
plt.plot(w, 20 * np.log10(abs(h)))

Bessel - Respuesta en Frecuencia



2-

Característica	FIR	IIR
Estabilidad	Siempre estables	Pueden volverse inestables
Retardo de fase	Fase lineal (ideal para audio/imágenes)	Fase no lineal (introduce distorsión de fase)
Eficiencia computacional	Requiere más coeficientes y operaciones	Requiere menos coeficientes (más eficientes)
Implementación	Solo con ceros	Tiene ceros y polos
Aplicaciones típicas	Procesamiento de señales críticas en fase	Filtros analógicos convertidos a digitales

✓ -3

```

1 # Instalación de dependencias
2 !pip install yt-dlp soundfile
3 !apt install ffmpeg
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from scipy.signal import firwin, freqz, lfilter, remez
8 from IPython.display import Audio, display
9 from matplotlib.ticker import EngFormatter
10 import soundfile as sf
11 import os
12
13 # -----
14 # DESCARGAR Y PREPARAR AUDIO
15 # -----
16
17 def descargar_audio(url):
18     """Descarga audio desde YouTube y lo convierte a WAV"""
19     !yt-dlp -x --audio-format mp3 -o "audio_temp" "$url"
20     !ffmpeg -i audio_temp.mp3 -ac 2 -ar 44100 audio.wav -y
21     os.remove("audio_temp.mp3") # Eliminar el archivo mp3 temporal
22     return "audio.wav"
23
24 # URL de ejemplo (puedes cambiarla por tu canción preferida)
25 url_audio = "https://youtu.be/ApXoWvfEYVU?list=RDMM&t=22" # Ed Sheeran - Shape of You
26 nombre_archivo = descargar_audio(url_audio)
27
28 # Cargar el audio
29 x, fs = sf.read(nombre_archivo)
30 print(f"Frecuencia de muestreo: {fs} Hz, canales: {x.shape[1]}, duración: {x.shape[0]/fs:.2f} s")
31
32 # Seleccionar segmento de 10 segundos
33 ti = 30 # Comenzar en el segundo 30
34 tf = ti + 10
35 xs = x[int(ti*fs):int(tf*fs), :]
36
37 # Reproducir segmento original
38 print("\nSegmento de audio original:")
39 display(Audio(xs.T, rate=fs))
40
41 # -----
42 # CONFIGURACIÓN FILTROS FIR
43 # -----
44
45 N_fir = 101 # Número de coeficientes (taps)
46 Wn = [1000, 4000] # Banda pasante [Hz]
47 filtro = 'bandpass' # Tipo de filtro
48
49 # -----
50 # FUNCIONES AUXILIARES
51 # -----
52
53 def graficar_respuesta_frecuencia(w, h, title):
54     """Grafica la respuesta en frecuencia en dB"""
55     fig, ax = plt.subplots(figsize=(10, 5))
56     ax.plot(w, 20 * np.log10(np.maximum(abs(h), 1e-10)))
57     ax.set_title(title)
58     ax.set_ylabel("Amplitud [dB]")
59     ax.set_xlabel("Frecuencia [Hz]")

```